

שלב 1 - פרימיטיבים וגופים גיאומטריים

בשלב הראשון של הפרויקט נבנה את התשתית המתמטית והגיאומטרית שעליה יתבסס מנוע הרינדור בהמשך הקורס. נממש את הפרימיטיבים הגיאומטריים הבסיסיים (נקודות, וקטורים וקרניים), נגדיר מחלקות עבור גופים גיאומטריים שונים, ונבדוק את פעולות הפרימיטיבים באמצעות תוכנית בדיקה ראשית. חשוב להקפיד על מימוש מדויק ויעיל כבר בשלב זה, משום שכל החישובים הגיאומטריים בהמשך הפרויקט נשענים על מחלקות אלו.

לפני תחילת ביצוע ההנחיות יש ללמוד את המצגת של השלב. חלק מהמידע המוזכר בהנחיות מופיע במצגת הזו.

1. הגדרות מקדימות

לפני תחילת העבודה, יש להכין את סביבת הפיתוח לפרויקט בעזרת ההגדרות המפורטות בנספח A.

2. מבנה הפרויקט והחבילות

בפרויקט יוגדרו החבילות הבאות:

- primitives
- geometries
 - geometries.api
 - geometries.impl
- test

תפקיד החבילות:

תפקיד	חבילה
מחלקות היסוד של המערכת הגיאומטרית	primitives
מחלקות וממשקים אבסטרקטיים של גופים גיאומטריים	geometries.api
מימושים קונקרטיים של הגופים	geometries.impl
תוכנית הבדיקה הראשית של שלב 1	test
חלוקה זו מבוססת על עקרון Dependency Inversion Principle (DIP): הקוד יעבוד מול הממשקים והמחלקות האבסטרקטיות שב-api, בעוד המימושים נמצאים ב-impl.	

3. מודולים המסופקים לסטודנטים

בשלב זה מקבלים מספר מודולים מוכנים:

חבילה	מודול
primitives	Util.java
primitives	Double3.java
geometries.impl	Polygon.java
test	Main.java

מודולים אלו כבר ממומשים ונבדקו ולכן אין לשנות אותם. מומלץ לעבור על הקוד וללמוד ממנו:

- סגנון קוד
- מבנה תיעוד Javadoc
- שימוש בעקרונות DRY ו-immutability

4. עקרונות מימוש כלליים

אובייקטים בלתי נתונים לשינוי (Immutable Objects)

- כל האובייקטים של הפרימיטיבים ושל הגופים הגיאומטריים יהיו immutable. כלומר:
- כל השדות יוגדרו final
- אין מתודות שמשנות מצב
- כל פעולה מחזירה אובייקט חדש.
- גישה זו מפחיתה תקלות ומבטיחה יציבות בהמשך הפרויקט.

שימוש ב-final

- יש להקפיד על הכללים הבאים:
- כל מחלקה שאין ממנה ירשה תוגדר final
- במחלקות אב, מתודות שאין צורך לדרוס במחלקות יורשות יוגדרו final
- מטרה:
- למנוע דריסה בטעות
- לשמור על יציבות הארכיטקטורה

שימוש במחלקת Util

- בפרויקט זה אין להשוות מספרים ממשיים ל-0 באמצעות == או אופרטורים של השוואה יחסית, כגון < וכדומה. במקום זאת יש להשתמש במתודות:
- Util.isZero(...)
- Util.alignZero(...)
- מתודות אלו מטפלות בשגיאות עיגול של חישובי נקודה צפה.

עקרון DRY

- יש להימנע משכפול קוד. בפרט:
- יש להשתמש במתודות של Double3
- יש להשתמש במתודות קיימות במקום לחשב שוב
- אין להעתיק נוסחאות בין מחלקות

הנחיות נוספות

- יש להוסיף לכל המחלקות, השדות והמתודות תיעוד בפורמט JavaDoc – ראה במצגת השלב ובנספח B
- יש להקפיד על דרישות עיצוב קוד ומוסכמות שמות – ראה במצגת השלב ובנספח C
- אין להגדיר מתודות אחזור (גטרים) **בשלב הזה**
- **אין** להגדיר מתודות עדכון (סטרים) **בכלל**
- כל השדות פרטיים למעט שדות קבועים המוזכרים בהמשך
- דריסת מתודה השוואה (equals) לפי הכללים והקוד המופיע במצגת השלב
- דריסת מתודת תיעוד (toString) על מנת להציג את האובייקטים בצורה תמציתית אך נוחה (השימוש בעיקר לצורכי דיבוג)
- הגדרת משתנים מקומיים במידת הצורך מאחד הסיבות הבאות בלבד:
 - עיקרון DRY וביצועים (ביטוי שיש יותר משימוש אחד בו)
 - שיפור קריאות (מניעת ביטויים ארוכים מדי)

5. חבילת primitives

חבילה זו מכילה את מחלקות היסוד הגיאומטריות. המחלקות:

- Util - מסופקת ממומשת
- Double3 - מסופקת ממומשת
- Point
- Vector
- Ray

מחלקת Point

המחלקה מייצגת נקודה במרחב תלת-ממדי.

שדות:

- שדה xyz מטיפוס Double3
- שדה קבוע ZERO המייצג את ראשית הצירים

בנאים:

- Point(double x, double y, double z)
- Point(Double3 xyz)

מתודות שיש לממש:

- Vector subtract(Point other) - מחזירה את הווקטור מהנקודה השנייה לנקודה הנוכחית
- Point add(Vector vector) - מחזירה נקודה חדשה לאחר הזזת הנקודה בווקטור הנתון
- double distanceSquared(Point other) - מחזירה את ריבוע המרחק בין שתי נקודות
- double distance(Point other) - מחזירה את המרחק בין שתי נקודות, מימוש המתודה חייב להשתמש ב-distanceSquared

מחלקת Vector

ווקטור מייצג כיוון וגודל במרחב. המחלקה יורשת מ-Point.

בנאים:

- Vector(double x, double y, double z)
 - אין לאפשר יצירת ווקטור אפס, יש לזרוק IllegalArgumentException במצב כזה
- Vector(Double3 xyz)
 - אין לאפשר יצירת ווקטור אפס, יש לזרוק IllegalArgumentException במצב כזה

מתודות שיש לממש:

- add
- scale
- dotProduct
- crossProduct
- lengthSquared
- length
- distanceSquared
- distance
- normalize
- יש להשתמש בנוסחאות של אלגברה לינארית בלבד. אין להשתמש בנוסחאות טריגונומטריות.

מחלקת Ray

קו מייצגת חצי-ישר במרחב.

שדות:

- Point origin
- Vector direction

בנאי:

- Ray(Point origin, Vector direction) ווקטור הכיוון חייב להישמר מנורמל לכן הבנאי ינרמל את הווקטור לפני שמירתו

6. חבילת geometries

בחבילה זו יוגדרו הגופים הגיאומטריים.
החבילה מחולקת לשתי תת-חבילות:

- geometries.api
- geometries.impl

בנאים במחלקות:

- בכל המחלקות יהיה בנאי שמקבל נתונים מתאימים לאתחול כל השדות המתוארים
- בחלק מהמחלקות יהיו הנחיות פרטניות לגבי הבנאי(ים)
- מחלקות יורשות ממחלקת אב עם שדות – מזמנות את הבנאי של מחלקת האב על מנת לאתחל את הנתונים

מחלקות אבסטרקטיות Geometry

מחלקת בסיס לכל הגופים. מוגדרת בחבילה geometries.api.
בשלב הזה אין בנאי.

מתודה:

- Vector getNormal(Point point) - מתודה זו מחזירה את הנורמל לגוף בנקודה הנתונה

מחלקות אבסטרקטיות RadialGeometry

מחלקת בסיס לגופים בעלי רדיוס. מוגדרת בחבילה geometries.impl.
שדות:

- radius
- radiusSquared

בנאי:

- RadialGeometry(double radius), מאתחל את שני השדות

גופים גיאומטריים

יש להגדיר את המחלקות:

- Polygon - מסופקת ממומשת
- Plane
- Triangle
- Sphere
- Tube
- Cylinder

Plane

המחלקה מייצגת מישור.

שדות:

- Point point
- Vector normal

בנאים:

- Plane(Point p1, Point p2, Point p3)
 - בשלב הזה הבנאי ישמור את הנקודה בלבד
 - Plane(Point point, Vector normal)
 - בשלב זה הבנאי ישמור את הנקודה ואת הנורמל לאחד נרמול
- המתודה** getNormal תחזיר את הנורמל.

Triangle

יורש מ-Polygon. אין צורך להוסיף שדות.

Sphere

שדות:

- Point center
- double radius

Tube

שדות:

- Ray axis
- double radius

Cylinder
 יורש מ-Tube.
 שדה נוסף:
 • double height

מימושי מתודות getNormal
 בשלב זה:

מימוש	מחלקה
מחזירה את הנורמל	Plane
כבר ממומש	Polygon
לא מממשים כלל	Cylinder ו-Triangle
מחזירים null	Tube ו-Sphere

7. תוכנית הבדיקה

מצורפת תוכנית Main שמבצעת בדיקות עבור פעולות הפרימיטיבים ובניית הגופים.
 יש לוודא שהרצת התוכנית:

- אינה מדפיסה שורות המתחילות ב-ERROR
 - אינה זורקת חריגות
- תוכנית זו תשמש גם בשלבים הבאים של הפרויקט לבדיקת רגרסיה.

8. הגשה

העבודה מתבצעת בזוגות. יש להשתמש ב-Git ו-GitHub.
 שם המאגר: ISE5786_XXXX_YYYY, כאשר XXXX ו-YYYY הן ארבע הספרות האחרונות של תעודות הזהות.
 יש ליצור tag בשם: **PR01** על ה-commit האחרון, לעשות push של תג ל-GitHub, ולהגיש את הקישור לתג ב-GitHub.

נספח A – הגדרות בסביבות הפיתוח (IDE Configuration)

בקורס מומלץ להשתמש באחת משתי סביבות הפיתוח:

- **Eclipse**

- **IntelliJ IDEA**

מטרת ההגדרות בנספח זה היא להבטיח:

- סגנון קוד אחיד בין כל הסטודנטים
- מניעת בעיות פורמט
- ארגון אוטומטי של imports
- יצירת תיעוד Javadoc תקין

הגדרות אלו גם מפחיתות תקלות נפוצות במהלך העבודה בפרויקט.

A.1 עבודה ללא Java modules

הפרויקט בקורס אינו משתמש במנגנון Java modules. לכן כשאתם פותחים פרויקט Java חדש עבור הקורס:

- אין להשתמש במודולים בפרויקט
 - אין ליצור קובץ module-info.java
 - אין להשתמש בהגדרות module במערכת הבנייה
- הסיבה לכך היא ש-Java אינה מטרת הקורס, והשימוש במודולים (ובמיוחד תלויות בין מודולים) מוסיף מורכבות שאינה נדרשת בפרויקט. הקוד בפרויקט בנוי במבנה **packages רגיל בלבד**. בעת יצירת הפרויקט יש לוודא שלא נוצר הקובץ module-info.java:

IntelliJ IDEA	Eclipse
בחלון יצירת פרויקט אין לבחור פרויקט מסוג Java module	בחלון יצירת פרויקט יש להסיר את הסימון של: Create module-info.java

אם נוצר module-info.java בטעות ה-IDE עלול להציג שגיאות קומפילציה כגון:

- package primitives is not visible
- package geometries is not visible
- module does not read another module
- או שגיאות import בין החבילות

פתרון

1. למחוק את הקובץ module-info.java
 2. לבצע Rebuild Project
- לאחר מחיקת הקובץ הפרויקט יחזור למצב עבודה רגיל.

A.2 פירמוט קוד אוטומטי וארגון imports בעת שמירה

יש להפעיל פעולות אוטומטיות בעת שמירת קובץ. פעולות אלו יבצעו:

- פירמוט קוד (הזחות ורווחים)
 - ארגון imports
 - הסרת imports שאינם בשימוש
- כך נשמר סגנון קוד אחיד בפרויקט.
בנוסף, יש להגדיר שה-IDE **לא ישתמש ב-wildcard imports (*)**. יש להגדיר שהמעבר ל-* יתבצע רק כאשר יש לפחות **99 imports**, כך שבפועל לא ייווצרו wildcard imports.
כדאי לשמור על אחידות בין כל הסטודנטים בפרויקט. יש להגדיר סדר קבוע של imports. סדר ה-imports בפרויקט יהיה:
- ספריות Java הסטנדרטיות (java.*)
 - חבילות הפרויקט
- דוגמה:

```
import java.util.List;
import java.util.Objects;

import primitives.Point;
import geometries.impl.Sphere;
```

הפרדה בין הקבוצות מתבצעת באמצעות שורת רווח אחת.

IntelliJ IDEA	Eclipse
תפריט: File→Settings→Tools→Actions on Save יש לסמן: ✓ Reformat code ✓ Optimize imports מניעת wildcard imports תפריט: File →Settings→Editor→Code Style→Java→Imports יש להגדיר: Class count to use import with '*' = 99 Names count to use static import with '*' = 99 סדר imports קבוע תפריט: File→Settings→Editor→Code Style→Java→Imports בקטגוריית Import Layout יש להגדיר: <pre>import java.* (blank line) import all other imports</pre>	תפריט: Window→Preferences→Java→Editor→Save Actions יש לסמן: ✓ Perform the selected actions on save ✓ Format source code ✓ Organize imports מומלץ לבחור: Format all lines מניעת wildcard imports תפריט: Window→Preferences→Java→Code Style→Organize Imports יש להגדיר: Number of imports needed for '*' = 99 Number of static imports needed for '*' = 99 סדר imports קבוע תפריט: Window → Preferences → Java → Code Style → Organize Imports ברשימת Import order יש להגדיר: java חבילות הפרויקט ימוקמו אוטומטית לאחר מכן.

A.3 פירמוט ידני של הקוד

לעיתים נדרש לפרמט קוד ידנית באמצעות ה-formatter של סביבת הפיתוח. פעולה זו מתקנת באופן אוטומטי:

- הזחות
- רווחים סביב אופרטורים
- מבנה בלוקים וסוגריים
- סגנון כללי של הקוד
- קיצורי המקשים לפירמוט:

IDE **קיצור**
Ctrl+Shift+F Eclipse
Ctrl+Alt+L IntelliJ IDEA

ניתן להפעיל פירמוט **על מספר** לבחור ב-IDE את התיקיה `src`. לאחר מכן יש להפעיל את קיצור הפירמוט. ה-IDE יבצע פירמוט על כל קבצי הקוד שבתיקיה `src`.

נ.ב. חשוב לבחור את תיקיית `src` בלבד ולא את כל הפרויקט, כדי להימנע מפירמוט קבצים שאינם קוד מקור, כגון קבצי תיעוד שנוצרים בתיקיה `doc`.

לפני יצירת תג ההגשה (tag) לפרויקט מומלץ לבצע:

- פירמוט לכל קוד המקור על ידי בחירת תיקיית `src`
- שמירת כל הקבצים
- `commit` ו-`push` של השינויים

פעולה זו מבטיחה שכל הקוד בפרויקט עומד במוסכמות הפורמט של הקורס.

A.4 הגדרת שורת רווח אחת (Blank Lines) ב-IntelliJ IDEA

כאשר אתם מוסיפים שורות רווח – בקורס נדרשת **שורת רווח אחת בלבד**. ב-IntelliJ ברירת המחדל היא **שתי שורות רווח**, ולכן יש לשנות זאת.

תפריט: File→Settings→Editor→Code Style→Java→Blank Line
יש להגדיר:

הגדרה	ערך
1	Around method
1	Around class
0	Before method body

A.5 קידוד קבצים UTF-8

כדי למנוע בעיות תווים בין מערכות שונות יש להגדיר קידוד קבצים אחיד ולכן יש להשתמש בקידוד UTF-8.

IntelliJ IDEA	Eclipse
תפריט: File→Settings→Editor→File Encodings יש להגדיר:	תפריט: Window→Preferences→General→Workspace יש לבחור: Text file encoding→UTF-8
Global Encoding → UTF-8 Project Encoding → UTF-8	

נספח B – הגדרות יצירת תיעוד Javadoc

בסיום כל שלב בפרויקט יש ליצור מעין אתר תיעוד Javadoc עבור כל הפרויקט. **מטרות:**

- בדיקת תקינות התיעוד
- איתור שגיאות
- יצירת תיעוד חיצוני של הקוד

B.1 הגדרות כלליות

בעת יצירת התיעוד יש להשתמש בהגדרות:

פרמטר	ערך
Visibility	private
Output folder	doc
Additional parameter	-quiet

B.2 B.2 תיקיית doc

התיעוד ייווצר בתיקייה doc בתוך תיקיית הפרויקט.

התיקייה אינה חלק מקוד המקור ואינה במעקב Git. לכן יש להוסיף לקובץ **.gitignore**. את השורה:

/doc/

B.3 יצירת אתר תיעוד Javadoc

IntelliJ IDEA	Eclipse
Tools → Generate JavaDoc : תפריט : הגדרות	Project → Generate Javadoc : תפריט : הגדרות (שימו לב שהגדרות הבאות מפורזות בין מספר חלונות של ההפעלה של יצירת Javadoc):
Scope: whole project	Scope: Whole project
Visibility: private	Visibility: private
Output directory: doc	Destination: doc
Other command line arguments: -quiet	VM options: -quiet

לאחר יצירת התיעוד יש לוודא שאין שגיאות.

נספח – C מוסכמות קוד (Coding Conventions)

מוסכמות קוד אחידות הן חלק מדרישות הקורס.

C.1 מוסכמות שמות

רכיב	פורמט
Classes	PascalCase
Methods	camelCase
Variables	camelCase
Parameters	camelCase
Instance fields	_fieldName
Static fields	s_fieldName
Constants	UPPER_CASE

C.2 עיצוב

עבור סוגריים מסולסלות יש להשתמש בסגנון Java:

```
if (condition) {
    doSomething();
} else {
    doSomethingElse();
}
```

יש להקפיד על:

- הזחה תקינה
- רווחים סביב אופרטורים
- שורת רווח אחת בלבד ברצף
- ראה בעניין הפירמוט בנספח A.

C.3 כתיבת תיעוד Javadoc

יש לכתוב Javadoc לפני:

- כל מחלקה
- כל שדה
- כל מתודה

אין לכתוב Javadoc עבור:

- מתודות @Override
- מימושים של מתודות interface

התיעוד חייב להיכתב באנגלית בלבד.

יש ליצור ולכתוב תיעוד Javadoc מיד לאחר הגדרת כותרת המחלקה, מיד לאחר הגדרת כותרת המתודה, מיד לאחר הגדרת שדה.

C.4 עקרונות תכנון חשובים בפרויקט

Immutability

- כל השדות final
- אין setters
- כל פעולה מחזירה אובייקט חדש

DRY

- אין שכפול קוד
- שימוש במתודות קיימות

Efficiency

- הימנעות מיצירת אובייקטים זמניים מיותרים